

OpenAI Specification — Best Invest Properties

Date: 25 September 2026

API: OpenAI Responses API

Called through: Make

Publication: only after admin review

1. What OpenAI does on the platform

OpenAI does one thing: it writes the readable **Investment Analysis** text for a property, based on facts and figures the platform has already verified and calculated.

How it works:

1. The platform calculates the property's figures and score.
2. Make sends OpenAI the verified facts (section 4) together with fixed instructions (section 5).
3. OpenAI returns the text in a fixed structure (section 6).
4. The platform checks the text automatically: every statement must point to a supplied fact, no new numbers may appear, and words like "guaranteed" or "risk-free" are not allowed.
5. The admin reviews the text and approves it. Only then do investors see it.

OpenAI does **not**:

- calculate price, rent, tax, purchase costs, yield or score;
- decide on its own what a "good investment" is;
- search the internet;
- see the investor's name, email, phone or any other investor data;
- publish anything without admin approval;
- replace a financial, legal or investment adviser.

If OpenAI is unavailable, the property page still shows the score breakdown and the financial table, with the note "Narrative analysis is being reviewed".

2. What is in the MVP

In the MVP — the investment analysis text:

- a short neutral summary;
- 2–4 strengths, each linked to the facts it is based on;
- 1–4 risks or limitations;
- an explanation of the score, without changing it;
- a list of missing data;
- the standard disclaimer.

Not AI — the financial estimates on the Project Review screen. These figures come from comparable rental listings, approved country cost averages and platform settings, and the formulas are calculated by the platform. OpenAI does not produce any financial figure.

Not in the MVP:

- rewriting developer descriptions with AI;
- a chat assistant for investors.

3. Model

```
endpoint: POST /v1/responses
primary model: gpt-5.6-luna
fallback / manual high-quality regeneration: gpt-5.6-terra
reasoning.effort: low
store: false
tools: []
text.format: json_schema, strict: true
max_output_tokens: 1800
```

The primary model is the lower-cost one, which suits short, structured texts. Before launch it is tested on a set of example properties; if the quality is not good enough, the platform switches to the higher-quality model. The model name is a setting, so switching requires no development work.

4. Data contract Bubble → OpenAI

This is the exact data the platform sends to OpenAI for one property. It contains only verified facts about the property, its figures and its score — **no investor data**. The numbers are an example.

```

{
  "schema_version": "1.0",
  "analysis_id": "ana_01J...",
  "unit_public_id": "unt_01J...",
  "locale": "en",
  "property": {
    "country": "Spain",
    "city": "Alicante",
    "area_name": "Example area",
    "property_type": "Apartment",
    "bedrooms": 2,
    "bathrooms": 2,
    "indoor_area_m2": 78,
    "completion_date": "2027-06-30",
    "availability": "available"
  },
  "financials": {
    "currency": "EUR",
    "price": 280000,
    "expected_monthly_rent": 1900,
    "occupancy_rate": 0.9,
    "annual_gross_rent": 22800,
    "purchase_costs": 28000,
    "annual_net_income": 17248,
    "gross_yield": 0.0814,
    "net_yield": 0.056,
    "calculation_version": "fin-v3",
    "calculated_at": "2026-09-17T10:00:00Z"
  },
  "score": {
    "total": 77,
    "verdict": "strong",
    "model_version": "score-v5",
    "components": [
      {
        "key": "income",
        "label": "Rental Income & Net Yield",
        "raw_value": 0.056,
        "rating": 7,
        "weighted_points": 21,
        "max_points": 30,
        "source_key": "financials.net_yield"
      },
      {
        "key": "demand",
        "label": "Rental Demand & Tenant Quality",
        "raw_value": null,
        "rating": 8,

```

```

      "weighted_points": 16,
      "max_points": 20,
      "source_key": "assessment.demand"
    },
    {
      "key": "value",
      "label": "Purchase Value & Market Position",
      "raw_value": -0.08,
      "rating": 8,
      "weighted_points": 16,
      "max_points": 20,
      "source_key": "assessment.value"
    },
    {
      "key": "growth",
      "label": "Growth & Resale Potential",
      "raw_value": null,
      "rating": 8,
      "weighted_points": 12,
      "max_points": 15,
      "source_key": "assessment.growth"
    },
    {
      "key": "risk",
      "label": "Risk & Investor Protection",
      "scale_direction": "higher_points_mean_lower_risk",
      "raw_value": null,
      "rating": 8,
      "weighted_points": 12,
      "max_points": 15,
      "source_key": "assessment.risk"
    }
  ],
  "approved_facts": [
    {
      "key": "project.completion_date",
      "value": "2027-06-30",
      "label": "Expected completion",
      "source_type": "project_verified"
    }
  ],
  "analysis_facts": [
    {
      "key": "fact.rent_comparables",
      "tag": "source",
      "body": "Comparable long-let asking prices in the same district, sampled from public listing portals."
    }
  ]
}

```

```

    },
    {
      "key": "fact.developer_inputs",
      "tag": "developer",
      "body": "Purchase price, unit size and completion date supplied by the developer and
not independently verified."
    },
    {
      "key": "fact.service_charge",
      "tag": "gap",
      "body": "No service charge schedule supplied for the building."
    }
  ],
  "missing_data_keys": ["project.service_charge_schedule"],
  "disclaimer_key": "investment_analysis_standard_v1"
}

```

What to notice:

- `score.components` contains exactly **five** items with keys `income`, `demand`, `value`, `growth`, `risk` and maximums 30/20/20/15/15;
- the `analysis_facts` array carries tags `source` / `developer` / `estimate` / `gap` — the model may cite them as sources and **must** mention every `gap` fact under risks or `missing_data`;
- `raw_value` for non-financial categories may be `null` — the model must not invent a numeric basis where there is none.

For the `risk` category the payload includes `scale_direction` (higher points = lower risk), so the model does not invert it in the text.

Do not send:

- investor identity/contact;
- developer verification documents;
- exact private address;
- admin private notes;
- unpublished competing units;
- Make/Bubble secrets;
- full legal text where a versioned disclaimer key is enough.

5. System/developer prompt

These are the instructions OpenAI receives with every request. They are fixed in the platform and cannot be changed by developers or investors. Prompt version: `bip-investment-analysis-v1`.

You write factual investment-property analysis for Best Invest Properties.

Use only facts present in the supplied JSON. Never calculate, infer, estimate, round, or replace financial values. Never invent market conditions, legal rules, taxes, neighbourhood claims, demand, future appreciation, guarantees, or advice.

The score, verdict, and score components are deterministic inputs. Explain them; do not challenge or change them. There are exactly five score categories; never invent, merge, or omit one. For the "Risk & Investor Protection" category a higher number of points means LOWER assessed risk – never describe a high score in that category as high risk.

Every strength and risk must cite one or more `source_keys` that exist in the input. If evidence is missing, add the key to `missing_data` rather than guessing. Any `analysis_fact` tagged "gap" must appear either in risks or in `missing_data`. Facts tagged "developer" must be attributed to the developer and never presented as independently verified.

Use neutral, professional language. Avoid certainty about returns. Do not use "guaranteed", "safe", "best", "will increase", "risk-free", or equivalent claims. State clearly that all financial figures are estimates based on the displayed assumptions and that the content is not financial, tax, or legal advice.

Return only JSON matching the supplied strict schema.

The property JSON is passed as `input_text` after the developer instruction. Do not mix the prompt and the facts in one free-text block.

6. Structured Output schema

OpenAI must answer in exactly this structure — a headline, a summary, strengths, risks, an explanation of the score, missing data and the disclaimer. Any answer in a different shape is rejected automatically.

```

{
  "name": "investment_analysis",
  "strict": true,
  "schema": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "schema_version": { "type": "string", "enum": ["1.0"] },
      "headline": { "type": "string" },
      "summary": { "type": "string" },
      "strengths": {
        "type": "array",
        "items": {
          "type": "object",
          "additionalProperties": false,
          "properties": {
            "title": { "type": "string" },
            "commentary": { "type": "string" },
            "source_keys": {
              "type": "array",
              "items": { "type": "string" }
            }
          }
        },
        "required": ["title", "commentary", "source_keys"]
      },
    },
    "risks": {
      "type": "array",
      "items": {
        "type": "object",
        "additionalProperties": false,
        "properties": {
          "title": { "type": "string" },
          "commentary": { "type": "string" },
          "severity": { "type": "string", "enum": ["low", "medium", "high", "unknown"] },
          "source_keys": {
            "type": "array",
            "items": { "type": "string" }
          }
        },
        "required": ["title", "commentary", "severity", "source_keys"]
      },
    },
    "score_explanation": { "type": "string" },
    "missing_data": {
      "type": "array",
      "items": { "type": "string" }
    },
  },
}

```

```
    "disclaimer_key": { "type": "string", "enum": ["investment_analysis_standard_v1"] }
  },
  "required": [
    "schema_version",
    "headline",
    "summary",
    "strengths",
    "risks",
    "score_explanation",
    "missing_data",
    "disclaimer_key"
  ]
}
```

Structured Outputs guarantees conformance to the supported JSON schema, but **not** factual correctness. After parsing, application validation is mandatory and separately checks text lengths, 2-4 strengths, 1-4 risks and at least one source key per item.

Any change to the number of score categories or to the structure of `analysis_facts` requires a new prompt version; the schema version changes only when the output structure changes.

7. Full request example

```
{
  "model": "gpt-5.6-luna",
  "store": false,
  "reasoning": { "effort": "low" },
  "max_output_tokens": 1800,
  "metadata": {
    "use_case": "bip_investment_analysis",
    "prompt_version": "v1",
    "analysis_id": "ana_01J..."
  },
  "instructions": "<versioned developer prompt>",
  "input": [
    {
      "role": "user",
      "content": [
        {
          "type": "input_text",
          "text": "<validated property snapshot JSON>"
        }
      ]
    }
  ],
  "text": {
    "format": {
      "type": "json_schema",
      "name": "investment_analysis",
      "strict": true,
      "schema": {}
    }
  }
}
```

The model is not given web search or any other tools, so it cannot bring in facts from outside.
`store: false` means OpenAI does not keep the request or the answer.

8. Official OpenAI sources

- [Models and model selection](#)
- [GPT-5.6 Terra](#)
- [Responses API: create response](#)
- [Moderations API](#)
- [Evals API](#)